

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND
MANAGEMENT**

**A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)
BALMATTA, MANGALORE**

LOW LEVEL DESIGN (LLD)

Multi-Cloud Hybrid Strategy for Agriculture

SUBMITTED BY:

Muhammed Umer Sufaidh

III Year BCA

(Artificial Intelligence, Cloud Computing and DevOps)With IBM&TCS

REGISTER NUMBER: 23BCAICD085

UNDER THE GUIDANCE OF

Ms.Sindhu

[Faculty Guide]

INDUSTRY IBM MENTOR: Sumit Kumar Shukla

DEPARTMENT OF COMPUTER SCIENCE

**IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD
OF THE DEGREE OF**

BACHELOR OF COMPUTER APPLICATIONS

MAY 2026

LOW LEVEL DESIGN (LLD)

Table of Contents

- 1. Introduction 4
 - 1.1 Scope of the Document 4
 - 1.2 Intended Audience 4
 - 1.3 System Overview 4
- 2. System Design 5
 - 2.1 Application Design / Module Diagram 5
 - 2.2 Process Flow 5
 - 2.3 Information Flow 6
 - 2.4 Component Design 6
 - 2.5 Key Design Points 8
 - 2.6 API Catalogue 8
- 3. Data Design 9
 - 3.1 Data Model 9
 - 3.2 Data Access Mechanism 9
 - 3.3 Data Retention 9
 - 3.4 Data Migration 10
- 4. Interfaces 11
- 5. State and Session Management 12
- 6. Caching 13
- 7. Non-Functional Requirements 14
 - 7.1 Security Aspects 14
 - 7.2 Performance Aspects 14
- 8. References 14

1. Introduction

1.1 Scope of the Document

This Low Level Design (LLD) document explains the internal working of the Multi-Cloud Hybrid Backup System for Agriculture (PRJN26-055). It covers the detailed implementation of each Python module, function-level logic, request/response flow, file handling mechanisms, failover control flow, error handling behavior, and session management. This document complements the HLD by providing implementation-level detail sufficient for a developer or tester to understand and reproduce the system.

1.2 Intended Audience

- Developers implementing or extending the backup system
- Testers designing test cases for failover, logging, and cloud upload verification
- Maintainers troubleshooting backup failures or credential errors
- University reviewers and industry mentors evaluating implementation quality

1.3 System Overview

The system consists of two Python files and supporting artifacts:

File	Language / Framework	Role
backup.py	Python 3 (stdlib + boto3)	Core backup engine, failover logic, sample data generator
app.py	Python 3 + Streamlit	Web dashboard UI for triggering and monitoring backups
backup_log.txt	Plain text	Runtime log file (appended on each run)
last_backup_summary.json	JSON	Persisted result of the most recent backup run
agriculture_data/	Directory	Source folder containing agriculture plain-text files
local_backup/	Directory	Destination for timestamped local backup copies

2. System Design

2.1 Application Design – Module Diagram

The system is organized into the following logical modules within backup.py:

Module / Function	Location	Responsibility
CONFIG (dict)	backup.py (global)	Centralized configuration: paths, credentials, region, log file
logging setup	backup.py (global)	Dual-handler logger: FileHandler + StreamHandler
backup_local()	backup.py	Copies source directory to local timestamped folder
backup_google_drive()	backup.py	Copies files to local Google Drive Desktop sync folder
backup_aws_s3()	backup.py	Uploads files to AWS S3 bucket using boto3
run_backup_with_failover() ()	backup.py	Orchestrates the three backup steps, logs results, writes summary
create_sample_agriculture_data()	backup.py	Generates five test agriculture files for demo/testing
Streamlit Dashboard	app.py	UI layer: imports backup.py and wraps functions in button events

2.2 Process Flow

Detailed step-by-step execution flow for a full backup run (e.g., button 1 in Streamlit or CLI choice 1):

Phase 1 – Initialization

- Python interpreter loads backup.py, initializes CONFIG dict and logging module
- FileHandler opens backup_log.txt in append mode; StreamHandler connects to stdout
- User triggers execution: via Streamlit button click or CLI menu choice

Phase 2 – Sample Data Generation (if chosen)

- create_sample_agriculture_data() called: creates ./agriculture_data/ if absent
- Writes five .txt files with hardcoded agriculture content using Python open() with 'w' mode

Phase 3 – Local Backup

- run_backup_with_failover() logs header banner
- Calls backup_local(source_dir, local_backup_dir)
 - Generates timestamp string: datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
 - Constructs dest path: os.path.join(local_backup_dir, 'backup_{timestamp}')
 - Calls shutil.copytree(source_dir, dest) – raises exception if source missing
 - Counts backed-up files using os.walk(); logs count; returns True

- On any exception: logs error message; returns False

Phase 4 – Google Drive Backup

- Calls `backup_google_drive(source_dir, folder_id, credentials_path)`
 - Creates target dir `G:\My Drive\AgriBackup` using `os.makedirs(exist_ok=True)`
 - Iterates `source_dir` with `os.walk()`; for each file calls `shutil.copy2(src, dest)`
 - Increments counter; logs each filename; returns True on completion
 - On any exception: logs error; returns False

Phase 5 – AWS S3 Backup

- Calls `backup_aws_s3(source_dir, bucket_name, aws_access_key, aws_secret_key, region)`
 - Imports boto3 inside the function (import failure caught as `ImportError`)
 - Creates boto3 S3 client with explicit credentials
 - Iterates source files; for each: constructs `s3_key = 'agriculture_backup/{timestamp}/{filename}'`
 - Calls `s3.upload_file(filepath, bucket_name, s3_key)` per file
 - Per-file exceptions (`NoCredentialsError`, `ClientError`) logged as `WARNING`; appended to `failed_files` list
 - Returns True if at least one file uploaded or all succeed; logs count

Phase 6 – Summary and Persistence

- results dict populated: `{local: bool, google_drive: bool, aws_s3: bool, timestamp: ISO string}`
- Summary banner logged to `backup_log.txt`
- If all three are False: `log.critical()` alert raised
- `json.dump(results)` written to `./last_backup_summary.json` (overwrite)

2.3 Information Flow

JSON structure of `last_backup_summary.json` after a successful run:

```
{
  "local": true,
  "google_drive": true,
  "aws_s3": true,
  "timestamp": "2026-04-26T02:06:53.648080"
}
```

Log file entry format (`backup_log.txt` sample):

```
2026-04-26 02:06:50,321 [INFO] [LOCAL] Starting backup to:
./local_backup/backup_20260426_020650
2026-04-26 02:06:50,890 [INFO] [LOCAL] \u2705 Success! 5 file(s) backed up to
./local_backup/backup_20260426_020650
2026-04-26 02:06:51,102 [INFO] [GDRIVE] Copied: crop_report_april2026.txt
2026-04-26 02:06:53,210 [INFO] [AWS S3] Uploaded: soil_analysis.txt \u2192
s3://agri-backup-umer-2026/...
```

2.4 Component Design

2.4.1 CONFIG Dictionary (backup.py)

All configurable parameters are stored in a single global dict for easy maintenance:

Key	Example Value	Purpose
source_dir	./agriculture_data	Directory of agriculture files to back up
local_backup_dir	./local_backup	Root folder for timestamped local backup copies
gdrive_folder_id	13FzUzFp5199i9O...	Google Drive folder ID (used for API reference)
gdrive_credentials	C:\...\service_account.json	Path to Google service account key file
s3_bucket_name	agri-backup-umer-2026	AWS S3 target bucket name
aws_access_key	AKIA3ALVEI3I...	AWS IAM access key ID
aws_secret_key	qyYEEvHuh...	AWS IAM secret access key
aws_region	ap-south-1	AWS region for S3 bucket (Mumbai)
log_file	./backup_log.txt	Path to the runtime log file

2.4.2 backup_local() – Detailed Logic

Function signature: backup_local(source_dir: str, backup_dir: str) -> bool

Step	Code Action	Notes
1. Timestamp	<code>datetime.datetime.now().strftime('%Y%m%d_%H%M%S')</code>	Unique folder name per run
2. Build dest path	<code>os.path.join(backup_dir, 'backup_{timestamp}')</code>	e.g. ./local_backup/backup_20260426_020650
3. Log start	<code>log.info('[LOCAL] Starting backup to: {dest}')</code>	Entry written to both file and terminal
4. Check source	<code>if not os.path.exists(source_dir): raise FileNotFoundError</code>	Raises descriptive exception if missing
5. Copy tree	<code>shutil.copytree(source_dir, dest)</code>	Recursive directory copy to new location
6. Count files	<code>sum(len(files) for _, _, files in os.walk(dest))</code>	Verifies files were actually copied
7. Return True	<code>log.info(success msg); return True</code>	Signals success to failover controller
8. Exception	<code>except Exception as e: log.error(e); return False</code>	Any error caught; False returned

2.4.3 backup_google_drive() – Detailed Logic

Function signature: backup_google_drive(source_dir: str, folder_id: str, credentials_path: str) -> bool

Note: This implementation uses the Google Drive Desktop sync folder (local path G:\My Drive\AgriBackup) rather than the Google Drive REST API. The Google Drive Desktop application handles cloud synchronization automatically.

Step	Code Action	Notes
1. Define target	<code>gdrive_sync_dir = r'G:\My Drive\AgriBackup'</code>	Fixed local sync path for Drive Desktop
2. Create folder	<code>os.makedirs(gdrive_sync_dir, exist_ok=True)</code>	Creates folder if not present
3. Walk source	<code>for root, dirs, files in os.walk(source_dir)</code>	Traverses all subdirectories
4. Copy each file	<code>shutil.copy2(filepath, dest_path)</code>	Preserves metadata (timestamps, permissions)
5. Log & count	<code>log.info('[GDRIVE] Copied: {filename}')</code>	Per-file entry for auditability
6. Return True	<code>log.info(success count); return True</code>	Drive Desktop syncs asynchronously after this
7. Exception	<code>except Exception as e: log.error(e); return False</code>	Covers path errors, permission errors

2.4.4 backup_aws_s3() – Detailed Logic

Function signature: `backup_aws_s3(source_dir: str, bucket_name: str, aws_access_key: str, aws_secret_key: str, region: str) -> bool`

Step	Code Action	Notes
1. Import boto3	<code>import boto3 (inside function)</code>	Lazy import; ImportError caught separately
2. Timestamp	<code>datetime.datetime.now().strftime('%Y%m%d_%H%M%S')</code>	Used in S3 key prefix for versioning
3. Create client	<code>boto3.client('s3', aws_access_key_id=..., aws_secret_access_key=..., region_name=...)</code>	Explicit credential auth (not IAM role)
4. Walk source	<code>for root, dirs, files in os.walk(source_dir)</code>	Iterates all files recursively
5. Build S3 key	<code>s3_key = f'agriculture_backup/{timestamp}/{filename}'</code>	Logical path in S3 bucket
6. Upload file	<code>s3.upload_file(filepath, bucket_name, s3_key)</code>	boto3 multipart threshold: 8MB (default)
7. Per-file error	<code>except (NoCredentialsError, ClientError): log.warning; append to failed_files</code>	Partial failures don't stop the loop
8. Return True	<code>log uploaded count; return True</code>	Returns True even if some files failed
9. ImportError	<code>log.error('pip install boto3'); return False</code>	Guides user to fix missing dependency

2.4.5 run_backup_with_failover() – Orchestration Logic

This function is the main entry point called by both app.py and the CLI `__main__` block.

- Initializes results dict: {local: False, google_drive: False, aws_s3: False, timestamp: ISO string}
- Calls `backup_local()` → stores result in `results['local']`
- Regardless of local result, calls `backup_google_drive()` → stores result in `results['google_drive']`
- Regardless of Drive result, calls `backup_aws_s3()` → stores result in `results['aws_s3']`
- Evaluates `any([results['local'], results['google_drive'], results['aws_s3']])` – if False, logs CRITICAL
- Calls `json.dump(results)` to write `last_backup_summary.json` (always, even on total failure)

2.5 Key Design Points

- All three backup destinations always run (not exclusive failover) – ensures maximum redundancy
- Each function is independently callable and testable – Streamlit exposes local-only test separately
- boto3 is imported lazily (inside `backup_aws_s3`) to prevent ImportError from breaking local backup
- All timestamps use Python `datetime.datetime.now()` – local system time, not UTC
- Google Drive backup uses Desktop sync approach, not Google Drive REST API or service account upload
- The CONFIG dict uses hardcoded credentials for the project prototype – production should use environment variables or secret managers
- `Streamlit st.spinner()` is used to prevent UI freezing during long backup operations

2.6 API Catalogue

Internal function interfaces (callable from app.py or CLI):

Function	Parameters	Returns	Side Effects
<code>backup_local(source_dir, backup_dir)</code>	str, str	bool	Creates folder in backup_dir; appends to log
<code>backup_google_drive(source_dir, folder_id, credentials_path)</code>	str, str, str	bool	Copies files to Drive sync dir; appends to log
<code>backup_aws_s3(source_dir, bucket, key, secret, region)</code>	str, str, str, str, str	bool	Uploads to S3; appends to log
<code>run_backup_with_failover(config)</code>	dict	None	Runs all 3 backups; writes JSON summary; appends log
<code>create_sample_agriculture_data()</code>	None	None	Creates <code>./agriculture_data/</code> with 5 test files

3. Data Design

Data Diagram – Entity and File Relationships:

Data Entity	Type	Storage	Description
crop_report_april2026.txt	Plain text file	agriculture_data/	Crop name, yield (tons/hectare), season, status
soil_analysis.txt	Plain text file	agriculture_data/	pH level, nitrogen, phosphorus, moisture %
weather_log.txt	Plain text file	agriculture_data/	Date, temperature (C), rainfall (mm), humidity (%)
irrigation_schedule.txt	Plain text file	agriculture_data/	Per-field watering days and times
pesticide_usage.txt	Plain text file	agriculture_data/	Chemical, quantity/acre, application date, next due
last_backup_summary.json	JSON object	./ (project root)	Result flags (local/gdrive/s3 bool) + ISO timestamp
backup_log.txt	Text log (append)	./ (project root)	All INFO/WARNING/ERROR/CRITICAL events across runs
local_backup/backup_{ts}/	Directory copy	local_backup/	Full copy of agriculture_data per backup run
S3: agriculture_backup/{ts}/{file}	S3 object	AWS S3 bucket	Per-file uploads with timestamp key prefix

3.1 Data Model

The last_backup_summary.json schema (actual example from last run on 2026-04-26):

```
{
  "local":      true,      // bool: local backup succeeded
  "google_drive": true,    // bool: Google Drive sync copy succeeded
  "aws_s3":     true,      // bool: S3 upload succeeded
  "timestamp":  "2026-04-26T02:06:53.648080" // ISO 8601 datetime
}
```

3.2 Data Access Mechanism

- Source file read: `os.walk(source_dir)` – returns (dirpath, dirnames, filenames) tuples
- Local copy: `shutil.copytree(src, dst)` – recursive, atomic directory-level copy
- Google Drive copy: `shutil.copy2(src, dst)` – individual file copy preserving metadata
- S3 upload: `boto3 s3.upload_file(Filename, Bucket, Key)` – per-file sequential upload
- Log write: `Python logging.FileHandler` opens `backup_log.txt` in 'a' (append) mode
- Summary write: `open(summary_path, 'w') + json.dump(results, f, indent=4)` – overwrites each run
- Summary read (app.py): `open(summary_file, 'r') + st.json(f.read())` – displays in Streamlit widget

3.3 Data Retention

- Local backups: Cumulative. Each run creates a new backup_{timestamp}/ subfolder. No automatic cleanup is implemented.
- Google Drive sync folder: Single copy per filename. Each run overwrites previous files with the same name (no versioning).
- AWS S3: Timestamped key prefixes create new versions for each run. S3 versioning and lifecycle policies are not configured in this prototype.
- backup_log.txt: Cumulative append. Never truncated. Grows with each run.
- last_backup_summary.json: Single record. Overwritten after each run.

3.4 Data Migration

No migration is currently required. The system runs as a standalone Python application. Future migration paths:

- Migrate source files from .txt to structured JSON or CSV for cloud-native analytics
- Replace last_backup_summary.json with an SQLite or PostgreSQL database for historical tracking
- Replace Google Drive Desktop sync with proper Google Drive API uploads using service account credentials
- Use AWS S3 Lifecycle Policies to auto-delete backups older than 30 days

4. Interfaces

4.1 Web Interface (app.py – Streamlit)

The Streamlit dashboard exposes the following UI elements:

UI Element	Streamlit Widget	Action
Current Configuration panel	st.expander()	Displays all CONFIG values (paths, bucket, region)
Button 1: Create Sample + Full Backup	st.button() + st.spinner()	Calls create_sample_agriculture_data() then run_backup_with_failover()
Button 2: Run Backup Only	st.button() + st.spinner()	Calls run_backup_with_failover(CONFIG)
Button 3: Test Local Backup	st.button() + st.spinner()	Calls backup_local() only; shows success/failure
Show Backup Log	st.text_area()	Reads and displays backup_log.txt content
Show Last Backup Summary	st.json()	Reads and displays last_backup_summary.json

4.2 Command-Line Interface (backup.py – __main__)

- Displays a numbered menu: 1 (Create data + full backup), 2 (Backup only), 3 (Local test only)
- Reads user input with input() and strips whitespace
- Routes to appropriate function based on string match ('1', '2', '3')
- Invalid inputs print an error message without crashing

4.3 Storage Interfaces

- Local FS: Python os and shutil stdlib – no external dependencies
- Google Drive: Google Drive Desktop app sync folder (local path, no API calls in current implementation)
- AWS S3: boto3 SDK v1.x – install via pip install boto3. Uses IAM access key authentication.

4.4 Logging Interface

- Logger name: __name__ (module-level logger)
- Format: '%(asctime)s [% (levelname)s] %(message)s'
- Handlers: logging.FileHandler(backup_log.txt, encoding='utf-8') + logging.StreamHandler()
- Levels: logging.INFO is the base level; WARNING, ERROR, CRITICAL used for escalation

5. State and Session Management

5.1 Stateless Execution Model

The backup engine (backup.py) is fully stateless between runs. No in-memory state is carried over from one execution to the next. All state is externalized to files:

State Element	Storage Medium	Persistence
Last backup result (success/failure)	last_backup_summary.json	Persists between runs (overwritten each time)
Historical backup events	backup_log.txt	Cumulative across all runs (append-only)
Backup file copies	local_backup/ directory	Persists between runs (one folder per run)
CONFIG dict	Python module memory	Exists only during runtime; reset on each execution

5.2 Session Flow (Streamlit)

- Each Streamlit button click is a new function call; no session variables are stored between button events
- st.spinner() provides visual feedback during blocking backup operations
- st.success(), st.error(), st.warning() messages appear inline below each button
- Log and summary displays are read from disk on each button click (no caching in Streamlit state)

Session Flow Diagram (conceptual): User clicks button → Streamlit calls Python function → Backup runs → Files written to disk → Streamlit reads files → Result displayed in UI

6. Caching

- No dedicated caching layer is implemented in this version
- CONFIG dict is loaded once at module import time and reused across all function calls within a single run
- boto3 S3 client is instantiated fresh on each backup_aws_s3() call; connection pooling is not implemented
- Python's import system caches the backup module after first import by app.py (standard Python behavior)
- Streamlit may cache static assets (CSS, JS) in the browser; no @st.cache_data decorators are used
- Optimization opportunity: Instantiate boto3 client once in CONFIG or as a module-level singleton to reduce authentication overhead across multiple runs in the same process

7. Non-Functional Requirements

7.1 Security Aspects

Security Concern	Current Implementation	Recommended Improvement
AWS credentials	Hardcoded in CONFIG dict (plaintext)	Use os.environ or ~/.aws/credentials file
Google service account key	Path hardcoded; file stored locally	Store in secure vault; exclude from git via .gitignore
No input validation	CLI input used directly as menu choice	Sanitize and validate all inputs in production
No auth on dashboard	Streamlit app open to any local user	Add Streamlit authentication or deploy behind VPN
S3 bucket access	IAM key with bucket-level access	Use least-privilege IAM policy scoped to bucket only

7.2 Performance Aspects

Operation	Current Behavior	Scalability Note
Local backup	Synchronous shutil.copytree()	Blocking; fast for small datasets (<100MB)
Google Drive copy	Sequential shutil.copy2() per file	Sync delay depends on Drive Desktop app speed
AWS S3 upload	Sequential boto3 upload_file() per file	Use ThreadPoolExecutor for parallel uploads at scale
Streamlit UI	Blocking during backup; spinner shown	Use threading or async for non-blocking UI
5-file test dataset	Total runtime < 5 seconds	Current scope well within performance budget

8. References

- Python shutil module documentation: <https://docs.python.org/3/library/shutil.html>
- Python os module documentation: <https://docs.python.org/3/library/os.html>
- Python logging module documentation: <https://docs.python.org/3/library/logging.html>
- Python datetime module documentation: <https://docs.python.org/3/library/datetime.html>
- Python json module documentation: <https://docs.python.org/3/library/json.html>
- AWS boto3 S3 Client reference: <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html>
- Google Drive Desktop App: <https://support.google.com/drive/answer/7329379>
- Streamlit API Reference: <https://docs.streamlit.io/library/api-reference>
- Source code: backup.py, app.py (PRJN26-055 project root)
- Project Google Drive folder: <https://drive.google.com/drive/folders/1D4zACdJ3684-sJjnyuYLz9bWaNcAM6KO>